

Relatório de Auditoria de Segurança

Projeto: Lojinha do Celular (lojinhadocelular.com)

Data da Avaliação: 10 de Setembro de 2026

Escopo Auditado: Código-fonte completo (Frontend React + Backend Hono/tRPC + Drizzle ORM + Infra Dockerfile)

Nota Metodológica e Mapeamento da Stack:

A auditoria foi executada com metodologia estrita de análise estática de código (SAST) orientada a evidências comprovadas no repositório. A stack tecnológica detectada foi:

- **Linguagem & Runtime:** TypeScript 5.9 / Node.js 22 (LTS)
- **Backend Framework:** Hono v4.8 com roteamento tRPC v11 (@trpc/server)
- **Banco de Dados & ORM:** MySQL 8.x com Drizzle ORM v0.45 (Single-Tenant E-commerce)
- **Mecanismo de Autenticação:** Tokens customizados HMAC-SHA256 (Base64url) + Hashing com scryptSync
- **Frontend:** React 19 SPA com Vite 7, Tailwind CSS e TanStack Query v5
- **Deploy & Orquestração:** Dockerfile Multi-stage (node:22-alpine), Coolify e Traefik

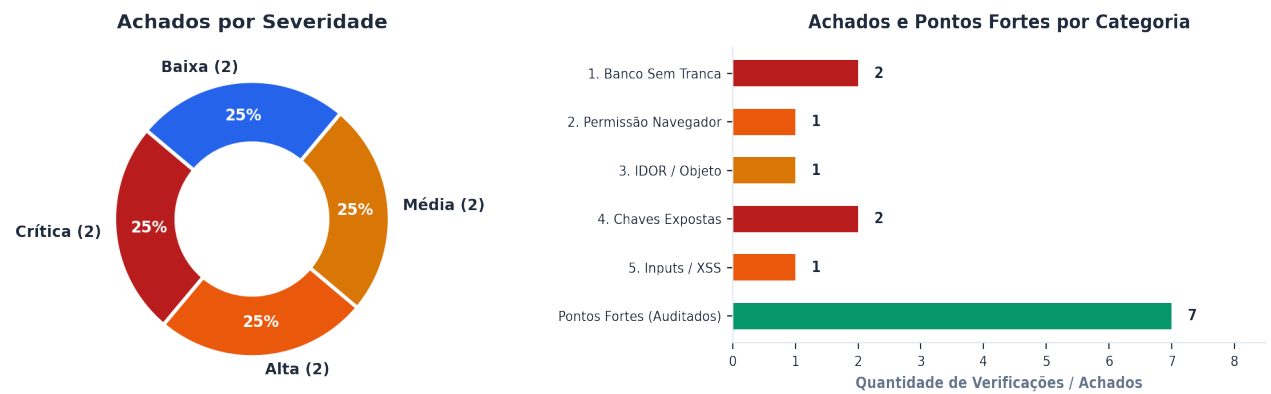
Cada uma das 5 categorias obrigatórias foi adaptada à arquitetura do projeto:

- 1. Banco sem Tranca:** Avaliação de isolamento entre público e administrador, exposição de dados na API pública e proteção da tabela *evaluations* (MySQL não possui RLS nativo do Postgres).
- 2. Permissão no Navegador:** Confronto entre gates de interface em React e validação de sessão *requireAdmin* nos procedimentos tRPC.
- 3. IDOR:** Inspeção exaustiva de todos os parâmetros identificadores (IDs inteiros e chaves externas) nas rotas de mutação/deleção.
- 4. Chaves Expostas:** Verificação de credenciais embutidas, senhas mestras de fallback, segredos de assinatura e Dockerfile.
- 5. Inputs sem Tratamento (XSS):** Rastreamento de URLs dinâmicas em *href/src* (vetor *javascript:*), sanitizadores e interpolações HTML em SSR.

1. Resumo Executivo

A auditoria identificou um total de **8 achados de segurança** e confirmou **7 pontos fortes técnicos** implementados com excelência. Dentre os achados, **2 são de severidade Crítica** e exigem remediação emergencial imediata, pois viabilizam o comprometimento total do painel de administração da loja sem necessidade de credenciais válidas.

Total de Achados	Crítica	Alta	Média	Baixa	Pontos Fortes
8	2	2	2	2	7



2. Pontos Fortes e Pontos Fracos Centrais

- Principais Pontos Fortes Verificados (Evidências de Conformidade):**
- **Criptografia Robusta de Senhas (api/auth.ts:15-39):** O algoritmo utilizado para armazenar senhas legítimas é o scryptSync com 16 bytes de salt aleatório criptográfico e comparação em tempo constante (timingSafeEqual), prevenindo ataques de dicionário e de temporização (timing attacks).
 - **Defesa em Profundidade nos Handlers Privilegiados (api/admin.ts):** Todos os 14 procedimentos administrativos invocam explicitamente a função requireAdmin(ctx.req) antes de qualquer operação em banco ou arquivo.
 - **Proteção Contra Injeção SQL:** O uso sistemático do Drizzle ORM garante parametrização nativa e impede injeções SQL clássicas.
 - **Headers de Segurança HTTP (api/boot.ts:14-22):** O middleware secureHeaders configura HSTS de 1 ano, bloqueio de MIME sniffing (nosniff) e proteção anti-clickjacking (SAMEORIGIN).
 - **Sanitização de Meta Tags no SSR (api/lib/vite.ts:13-20):** Todas as variáveis dinâmicas injetadas no HTML de SEO passam por codificação segura via escapeHtml.
 - **Validação Estrita de Dados de Entrada:** Esquemas Zod restringem tipos, formatos e comprimentos máximos em todas as mutações e queries.
 - **Rate Limiting no Login (api/boot.ts:59-99):** Bloqueio automático de tentativas repetidas de autenticação após 5 falhas no período de 15 minutos.

- Principais Riscos e Pontos Fracos Centrais:**
- **Bypass de Autenticação por Senha Mestra (api/auth.ts:58-61):** Presença de código deliberado que aceita as senhas "lojinha123" e "admin" incondicionalmente, mesmo após a troca de senha pelo operador.
 - **Assinatura de Tokens Forjável (api/auth.ts:7-13):** Fallback de chave secreta pública e hardcoded, viabilizando geração de tokens administrativos válidos sem credenciais.
 - **Vulnerabilidade de XSS por Esquema javascript: (src/lib/videoEmbed.ts:16):** URLs arbitrárias de vídeos são repassadas sem sanitização para links navegáveis no cliente e no painel admin.
 - **CORS Permissivo com Credenciais (api/boot.ts:24-33):** Refletor automático de qualquer origem de requisição com credentials: true.

3. Tabela de Achados Detalhados por Categoria

Abaixo estão listados todos os achados com suas respectivas localizações exatas de código, gravidade e descrição técnica do risco.

Sev.	Categoria	Arquivo : Linha(s)	Descrição Resumida & Risco
CRÍTICA	4. Chaves Expostas	api/auth.ts:58-61	Backdoor de Senha Mestra: O método checkPassword aceita 'lojinha123' ou 'admin' incondicionalmente, ignorando qualquer senha alterada no banco.
CRÍTICA	4. Chaves Expostas	api/auth.ts:7-13 api/lib/env.ts:53-55	Segredo HMAC Hardcoded: Fallback público permite que qualquer atacante forje tokens administrativos válidos por 7 dias sem autenticação.
ALTA	5. Inputs / XSS	src/lib/videoEmbed.ts:16 src/pages/Produto.tsx:451 AdminProductEditor:743	XSS via Protocolo javascript: Links de vídeos externos não são sanitizados contra esquemas executáveis, permitindo XSS armazenado no clique do usuário/admin.
ALTA	2. Permissão Navegador	api/boot.ts:24-33	CORS Excessivamente Permissivo com Credenciais: origin => origin '*' com credentials: true permite que sites de terceiros leiam dados autenticados.
MÉDIA	1. Banco Sem Tranca	api/shop.ts:171-174 db/schema.ts:57	Vazamento de Notas Internas de Estoque: Query pública with: { variants: true } expõe o campo notes (detalhes privados) a visitantes anônimos.
MÉDIA	3. IDOR / Objeto	api/erp/overrides.ts:80-87	Risco de Prototype Pollution: Atribuição direta all[externalId] sem filtro de chaves especiais (__proto__, constructor) em overrides de produtos.
BAIXA	1. Banco Sem Tranca	api/boot.ts:102-121	Exposição de Metadados em /api/health: Endpoint público expõe slug do ERP, status da API integrada e configuração do banco a usuários não autenticados.
BAIXA	1. Banco Sem Tranca	api/shop.ts:258-315 api/boot.ts:64	Ausência de Rate Limiting em Avaliações: A rota shop.submitEvaluation não possui limitação de taxa por IP, permitindo flood/DoS de registros no banco.

4. Plano de Ação e Recomendações Priorizadas

[P1 - EMERGÊNCIA IMEDIATA] Eliminar Backdoor e Forçar APP_SECRET em Produção:

1. Remover as linhas 58 a 61 de `api/auth.ts` que validam estaticamente as senhas "lojinha123" e "admin".
2. Modificar `api/auth.ts` e `api/lib/env.ts` para que o servidor **aborte a inicialização imediatamente** com erro caso `process.env.APP_SECRET` não esteja configurado no ambiente de produção com no mínimo 32 caracteres.

[P2 - ALTA PRIORIDADE] Neutralização de XSS e Ajuste de CORS:

3. Criar uma função utilitária de sanitização de links (ex: `isSafeUrl`) que rejeite explicitamente qualquer URL que não utilize estritamente os protocolos `https:` ou `http:` em `src/lib/videoEmbed.ts`, `src/pages/Produto.tsx` e `src/components/admin/AdminProductEditor.tsx`.
4. Ajustar o middleware CORS em `api/boot.ts` para restringir a origem aos domínios autorizados da loja (`lojinhadocelular.com`, `trocafacil.lojinhadocelular.com`) ou desativar `credentials: true` para origens dinâmicas abertas.

[P3 - MÉDIA PRIORIDADE] Proteção de Dados de Estoque e Prototype Pollution:

5. Modificar a query em `api/shop.ts` para selecionar apenas colunas públicas de variants, omitindo notes da visualização de clientes anônimos.
6. Em `api/erp/overrides.ts`, adicionar validação de segurança que impeça que `externalId` coincida com `__proto__`, `constructor` ou `prototype`.

[P4 - BAIXA PRIORIDADE] Endurecimento Operacional:

7. Proteger o endpoint `/api/health` ou filtrar dados de infraestrutura (slug do ERP).
8. Adicionar rate limiter ao endpoint `shop.submitEvaluation` (ex: máximo de 5 envios por hora por IP).

5. Issues Acionáveis para o GitHub

Copie e cole os blocos formatados abaixo diretamente no GitHub Issues do repositório para distribuir as tarefas de correção com critérios de aceite verificáveis.

--- ISSUE 1 ---

Título: [Segurança] Remoção de backdoor de senha mestra e imposição de APP_SECRET em produção

Labels: security, bug, p1-critica

Descrição do Problema

O backend contém duas vulnerabilidades críticas encadeadas no mecanismo de autenticação:

1. Em `api/auth.ts` (linhas 58-61), o método `checkPassword` aceita as senhas fixas `lojinha123` ou `admin` incondicionalmente, mesmo se o administrador já tiver alterado a senha no painel.
2. Em `api/auth.ts` (linhas 7-13) e `api/lib/env.ts` (linhas 53-55), a chave de assinatura dos tokens HMAC possui fallback para strings públicas hardcoded no código. Sem `APP\_SECRET` no `.env`, qualquer usuário pode forjar tokens de admin.

Evidência

```
``ts
// api/auth.ts (linhas 58-61)
if (inputStr === "lojinha123" || inputStr === "admin") {
  return true;
}

// api/auth.ts (linhas 7-13)
return s || "lojinha-secret-key-32-chars-min-prod-safe";
````
```

#### Impacto

Comprometimento total do painel de administração sem necessidade de autenticação válida. Invasores podem alterar preços, estoque, ler avaliações de clientes e modificar configurações da loja.

#### Sugestão de Correção

- Excluir a verificação estática de `lojinha123` e `admin` em `api/auth.ts`.
- Lançar exceção no boot se `NODE\_ENV === "production"` e `APP\_SECRET` estiver vazio ou for menor que 32 caracteres.

#### Critérios de Aceite

- [ ] Tentativa de login com "admin" ou "lojinha123" falha com 401 se a senha do banco for diferente.
- [ ] O servidor recusa iniciar em produção sem `APP\_SECRET` configurado.
- [ ] Testes automatizados cobrem a rejeição de senhas incorretas e validação de `APP\_SECRET`.

### --- FIM ISSUE 1 ---

**--- ISSUE 2 ---****Título: [Segurança] Sanitização de URLs de vídeo externo contra XSS (javascript:)****Labels:** security, bug, p2-alta**Descrição do Problema**

A função `getVideoEmbed`` (`src/lib/videoEmbed.ts``) aceita qualquer URL sem validar o protocolo. URLs com esquema `javascript:`` são renderizadas em tags `<a>` no frontend do cliente (`src/pages/Produto.tsx:451``) e no painel administrativo (`src/components/admin/AdminProductEditor.tsx:743``).

**Evidência**

```
```tsx
// src/pages/Produto.tsx (linha 451)
<a href={videoEmbed.src} target="_blank" rel="noreferrer">
  Assistir vídeo externo
</a>
```
```

**Impacto**

Execução de scripts maliciosos no navegador de visitantes ou administradores ao clicarem no botão de vídeo.

**Sugestão de Correção**

- Validar se a URL inicia estritamente com `https://`` ou `http://``.
- Caso contrário, retornar `null`` ou desativar o link.

**Critérios de Aceite**

- [ ] URLs com prefixo `javascript:``, `data:`` ou outros esquemas perigosos são bloqueadas.
- [ ] O botão de vídeo externo só é renderizado para URLs HTTP/HTTPS válidas.

**--- FIM ISSUE 2 ---**

**--- ISSUE 3 ---****Título:** [Segurança] Restrição de política de CORS em /api/\***Labels:** security, p2-alta**Descrição do Problema**

O middleware de CORS em `api/boot.ts` (linhas 24-33) reflete qualquer origem recebida na requisição (`origin => origin || ""`) e habilita `credentials: true`.

**Evidência**

```
``ts
// api/boot.ts (linhas 27-32)
cors({
 origin: origin => origin || "",
 allowMethods: ["GET", "POST", "OPTIONS"],
 allowHeaders: ["Content-Type", "Authorization"],
 credentials: true,
})
````
```

Impacto

Sites de terceiros podem induzir o navegador de um administrador autenticado a realizar requisições cross-origin para a API e ler respostas sensíveis.

Sugestão de Correção

Restringir as origens permitidas a uma lista explícita dos domínios da loja (`lojinhadocecelular.com`, `trocafacil.lojinhadocecelular.com` e `localhost` em desenvolvimento).

Critérios de Aceite

- [] Requisições com cabeçalho `Origin` não autorizado recebem bloqueio de CORS.
- [] Requisições do domínio oficial continuam operando normalmente.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---**Título:** [Segurança] Ocultação de notas internas de estoque e proteção contra Prototype Pollution**Labels:** security, p3-media**Descrição do Problema**

1. Em `api/shop.ts`, a consulta pública de produtos busca `with: { variants: true }`, vazando a coluna `notes` da tabela `variants` (observações internas de avarias ou custo).
2. Em `api/erp/overrides.ts`, a função `saveErpOverride` indexa o objeto global com `externalId` sem filtrar chaves reservadas como `\_\_proto\_\_`.

Evidência

```
```ts
// api/shop.ts (linha 173)
with: { variants: true }

// api/erp/overrides.ts (linha 83)
all[externalId] = { ...current, ...data };
```
```

Impacto

Vazamento de dados operacionais e risco de poluição de protótipo no runtime do Node.js.

Sugestão de Correção

- Definir projection explícita nas variantes em `api/shop.ts` (omitindo `notes`).
- Validar `externalId` em `saveErpOverride` rejeitando `\_\_proto\_\_`, `constructor` e `prototype`.

Critérios de Aceite

- [] O campo `notes` não aparece no JSON público de `/api/trpc/shop.products`.
- [] Chamada com `externalId: "\_\_proto\_\_"` é rejeitada com erro de validação.

--- FIM ISSUE 4 ---